Data Mining and Knowledge Discovery (2025) 39:0
https://doi.org/10.1007/s10618-024-01073-4

A probabilistic model for API contract specification retrieval focusing on the openAPI standard

Saeyoung Moon[1] · **Gregor Kerr**[1] · **Fran Silavong**[1] · **Sean Moran**[1]

Received: 29 October 2023 / Accepted: 10 October 2024
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2024

Abstract
Designing a new API for a large-scale project requires developers to make strategic design choices that will allow the codebase to evolve sustainably. To create well-structured API components, developers can learn from existing APIs. However, the lack of standardized methods for comparing API designs often makes this learning process inefficient and challenging. To bridge this gap, we introduce API-Miner, which, to our knowledge, is one of the first engines to recommend API-to-API specifications. API-Miner retrieves relevant components from OpenAPI specifications-a widely recognized standard for describing web APIs. The engine introduces several key innovations: (1) novel techniques for processing and extracting critical information from OpenAPI specifications, (2) specialized feature extraction methods tailored to the technical domain of API specifications, and (3) a log-linear probabilistic model that integrates multiple signals to retrieve relevant and high-quality OpenAPI components based on a query specification. Through both quantitative and qualitative evaluations, API-Miner achieves a recall@1 of 91.7% and an F1 score of 56.2%, outperforming baseline models by 15.4 percentage points (pp) in recall@1 and 3.2 pp in F1. API-Miner enables developers to access relevant OpenAPI components from public or internal databases early in the API development cycle, facilitating the learning of best practices and the identification of potential redundancies. This tool helps streamline the development process and supports the creation of maintainable, high-quality APIs. The code for API-Miner is available at https://github.com/jpmorganchase/api-miner.

Keywords API contract · API specification · Structured document matching · Recommendation system

Extended author information available on the last page of the article

Published online: 13 November 2024

1 Introduction

The Web continues its rapid expansion, with over 2,000 new web APIs (Application Programming Interface) being created annually since 2015 [1]. A Web API serves as an interface, providing access to the functionalities of a software service via endpoints (i.e. URLs) (Sohan et al. 2015). When tasked with developing a new API or extending an existing one, developers often follow an 'API-first' approach ,[1] where they start by drafting a specification that documents the API's design, including the functionalities of each endpoint. This ensures that the API design is robust and coherent with the overall project before the developers begin writing the actual code.

Designing new API functionalities can be a complex and challenging task. To illustrate the existing challenges, let's consider an example inspired by discussions with web API developers at our organization. Imagine Jess, a software developer working on a Music Library API, who is tasked with adding functionality to retrieve album information for a song. Following the 'API-first' approach, she begins by drafting a new endpoint in the API's specification. While Jess can draft some sections without difficulty, she faces challenges such as determining the appropriate input parameters and exploring better implementation methods.

To assist developers like Jess in making informed design decisions, we developed API-Miner, an API-to-API Specification Recommendation Engine. This tool processes the user's draft of an endpoint specification (i.e. a specific component of a specification related to an endpoint) and retrieves relevant, high-quality examples from a database. In the example shown in Fig. 1, API-Miner might suggest an endpoint that retrieves artist information based on a song ID or highlight a redundant endpoint to inform Jess that similar functionality already exists. In essence, API-Miner supports developers throughout the API design process by offering progressively more relevant examples as they refine their specification drafts. For this study, we focus on recommending components from OpenAPI specifications [3], a widely adopted standard for documenting RESTful APIs (Fielding and Taylor 2000). This paper expands upon an earlier workshop version (Young Moon et al. 2024), providing a more comprehensive description of the methodology, including enhanced explanations of featurization and the handling of syntactically invalid specifications. Additionally, this version includes significant new content, such as additional technical diagrams and a deeper analysis and discussion, all of which contribute to a more thorough understanding of the study's findings and their implications.

The primary contributions of this paper include:

- *API-to-API Specification Recommendation Engine*: We developed a recommendation engine that retrieves relevant endpoint specifications based on a user's query. An 'endpoint specification' refers to the section of an OpenAPI specification corresponding to a specific endpoint, where relevant specifications are those that (1) offer additional content not present in the user's current specification

[1] https://swagger.io/resources/articles/adopting-an-api-first-approach/

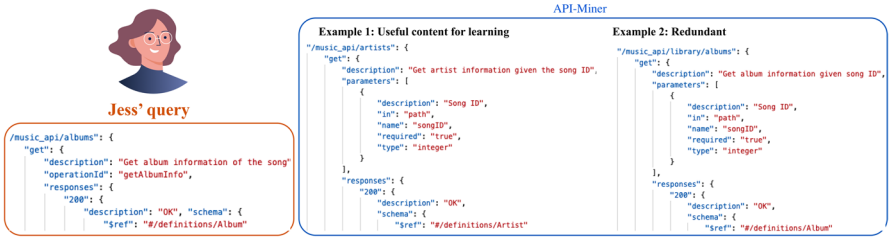


Fig. 1 Illustration of how API-Miner assists developers in the API design process

or (2) are redundant in structure and content with respect to the user's query. Additionally, API-Miner can suggest analogous endpoints for syntactically incomplete endpoint specifications (i.e. those missing required properties or with incomplete textual descriptions).

- *Feature Extraction*: We propose representing endpoint specifications as a *tree* to fully leverage the hierarchical relationships within the specification, extracting features from individual nodes and their paths. This approach addresses scenarios where the meaning of a token varies depending on its position in the hierarchy and its relationship to parent nodes. Our method extracts three sets of features to characterize endpoint specifications: (1) Tree Path tokens that capture hierarchical relationships, (2) Domain-Specific Natural Language Tokens, and (3) the API endpoint name itself.
- *Multi-Signal Ranking Model*: We introduce a probabilistic log-linear model that integrates multiple signals of similarity derived from our feature extraction methods, along with endpoint quality metrics, to generate a ranked list of relevant, high-quality endpoint specifications. We demonstrate that considering multiple sources of similarity enhances retrieval performance.

The remainder of this paper is structured as follows: Sect. 3 reviews related work in the API-to-API recommendation literature. Section 4 details the proposed API Specification Recommendation Engine, *API-Miner*. In Sect. 5, we evaluate API-Miner using quantitative retrieval tasks and a pooling-based user study. Finally, Sect. 6 presents conclusions and future research directions.

2 Background

An OpenAPI Specification (OAS) is a set of standardized rules for describing RESTful APIs. A RESTful API is a software architecture for building web services that adhere to the constraints of REST (i.e. *Representational State Transfer*) (Fielding 2000). There are some key requirements imposed by REST on any implementation, including: (1) *Uniform interface:* the server uses a standard format for data transfer involving HTTP and GET, PUT, POST, DELETE requests (Richardson and Ruby 2007), (2) *Statelessness:* the server handles client requests independently and stores no context between requests, (3) *Cacheability:* responses can be stored on the client

for efficiency, and (4) *Layered System:* intermediate servers may be involved in processing the request, and remain invisible to both the client and the server (Fielding 2000). These constraints, and others, allow a RESTful API to be lightweight, flexible, and maintainable, enabling interaction with any given client (e.g. web, mobile), which contributes to its popularity as a key component for building modern web services and distributed systems.

The OpenAPI specification for a RESTful API is typically written in JSON or YAML format and helps developers understand how they should interact with the API, what data they can get back, and how to leverage the API in building their applications [8]. The format is machine and human readable, language agnostic, and permits understanding of the key properties of the API without examining the underlying source code. The specification consists of several key parts, including but not limited to:

- *Endpoints*: which define the ways that a user can interact with the API; for example by sending a request to update or retrieve information.
- *Parameters*: describe what data and associated format is needed in a request to the server and includes path parameters, query parameters, headers, and request body.
- *Responses*: a description of what data will be sent back from the server in response to a request, including response codes, headers, and response bodies.
- *Data Models*: defines the structure and format of the data that is sent or received across the API.

There are numerous advantages of writing an OpenAPI specification for an API. The format is standardized, which allows enhanced interoperability and understanding of functionality across different platforms and systems. The API itself can be easily visualized and documented using tools like Swagger UI, and API testing, mocking, and code generation (in different programming languages) is straightforward [8].

The OpenAPI specification has evolved significantly over time, and at the time of writing, there are three main versions: v2.0 (formally known as Swagger v2.0), v3.0, and v3.1 (Initiative 2021). These versions differ in terms of their structure, features, and capabilities. Version 2.0 is the initial iteration of the specification originally developed by a company called Swagger (and subsequently donated to the OpenAPI initiative) that has been widely adopted and is popular for existing APIs [8]. Version 3.0 was released in 2017 and represents a significant revision, adding modality, flexibility, and extensibility, and supports both JSON and YAML formats. It simplifies the overall document structure compared to 2.0 by restructuring the specification into a more modular "components" section, separating reusable elements like schemas, responses, and parameters for better organization and reuse across the API definition. Version 3.0 additionally provides functionality for describing callbacks via a callback object for asynchronous APIs and webhook receivers, and includes an Examples Object to permit data samples to be provided for requests and responses, allowing users to better understand the expected data format. Version 3.1 is a more recent, incremental upgrade to the standard, backward compatible with

v3.0, encompassing technologies like WebSockets while refining existing features for improved usability and flexibility (Initiative 2021).

3 Related work

3.1 API retrieval systems

API-Miner is specifically designed to assist developers by identifying and retrieving endpoint specifications relevant to the endpoint they are currently developing. To our knowledge, there are no existing tools that provide retrieval of specific specification components based on a given endpoint. Consequently, we had to establish baselines using methodologies from related studies that focus on retrieving entire APIs in response to user queries, which we review in this section. Generally, APIs can be represented in one of two forms: (1) as keywords or textual descriptions, or (2) as full API specifications. The following review covers related work that addresses both types of representations.

APIs Represented by Keywords or Text: In the realm of web API recommendation systems designed for mashup development, APIs are often represented using keywords or descriptive text. These systems typically analyze API representations, often employing compatibility graphs, to identify a set of compatible APIs that developers can combine to build new software applications. Noteworthy contributions include Qi et al.'s weighted API correlation graph search (Qi et al. 2019), Fletcher's matrix factorization approach that integrates user preferences into API recommendations (Fletcher 2019), Wang et al.'s random walk technique on knowledge graphs (Wang et al. 2019), and Thung et al.'s TF-IDF-based vector semantic similarity approach (Thung et al. 2017). However, these methodologies were not directly applicable as baselines for API-Miner, as their primary focus differs considerably from the goals of our system.

APIs Represented by Specifications: Several existing web API retrieval systems represent APIs using formal specifications, such as the Web Application Description Language (WADL) (Hadley 2006), Web Services Description Language (WSDL) (Recommendation 2007), Universal Description, Discovery, and Integration (UDDI) [16], and OpenAPI Specification (Initiative 2021). Given that API-Miner is intended to process OpenAPI specification components, our baselines were constructed using methods that retrieve APIs based on these types of specifications.

Previous research has explored string matching techniques to assess the similarity of API specifications written in WADL and WSDL ((Khorasgani et al. 2011), (Bai and Liu 2011), (Wang and Stroulia 2003), (Aguilera et al. 2007)), with the aim of retrieving relevant APIs. In the context of OpenAPI specifications, Peng et al. (2018) proposed using Levenshtein distance to measure the similarity between descriptive (e.g., text) and functional (e.g., data-type) properties in service and resource levels of a specification. Similarity was evaluated using a weighted combination of these distances or by assigning relevance labels based on the calculated distances (e.g., low, medium, high). However, the string

matching approach has inherent limitations, primarily due to its inability to capture semantic meaning, which restricts its effectiveness in retrieving semantically similar specifications that do not contain exact string matches.

To address the shortcomings of string matching, various studies have utilized the textual descriptions in API specifications to compute semantic similarity and identify relevant APIs. For instance, Stroulia and Wang (2005) developed a search engine that evaluates the semantic and structural similarity of specifications written in WSDL. They calculated semantic similarity using WordNet embeddings of the natural language descriptions and structural similarity based on the data types and parameters specified. Their approach reported a recall@1 of 41% for semantic similarity and 22% for structural similarity, where recall@1 measures the system's success in retrieving the best match as its top recommendation. Although Wang et al.'s technique targets entire API specifications, it could not be directly used as a baseline for our approach, which focuses specifically on individual endpoint components. Nevertheless, we adapted their most effective API retrieval strategy to create a semantic similarity baseline for our study. It is worth noting that there has been limited exploration of this approach within the scope of OpenAPI specifications.

3.2 Vector semantics in low-resource domains

The specialized terminology found in API specifications introduces two key challenges for semantic representation learning: (1) the expanding and non-standardized nature of technical terms (such as acronyms), and (2) the limitations of standard pre-trained language model embeddings in capturing the semantic nuances of these technical terms. Although there have been efforts to fine-tune embeddings for low-resource domains, these approaches struggle with the rapidly growing vocabulary described in challenge (1) (Bahdanau et al. 2018)(Schick and Schütze 2019) (Pinter et al. 2017)(Moreo et al. 2021). Positive Pointwise Mutual Information (PPMI) is often employed to measure the likelihood of two events occurring together versus their independent occurrence. Prior research (Jungmaier et al. 2020) has suggested using PPMI for representing terms in low-resource contexts; however, this method is constrained by its inability to harness the rich contextual representations available from pre-trained language models. Therefore, in this work, we explore methods to create enriched text representations that leverage the contextual understanding provided by pre-trained language models, while still adequately representing technical terms.

4 Proposed methods

4.1 Vector representation of openAPI specifications

API-Miner takes endpoint specifications (i.e. components of an OpenAPI specification related to a specific endpoint) as input queries. Handling specification queries, rather than simple keywords or text descriptions, offers several benefits. First,

a specification can encapsulate complex design decisions, such as input parameters and responses, which are challenging to convey through textual descriptions alone. Second, developers typically refine their designs iteratively, often beginning with just the endpoints. The ability to repeatedly query using an evolving specification draft enhances their productivity and efficiency.

OpenAPI specifications can be structured in JSON or YAML formats, containing sections that comprehensively describe web APIs in a standardized manner [3]. Generally, a specification can be divided into three main sections: (1) the 'info' section provides general information about the API, (2) the 'paths' section includes endpoint specifications that detail the functionalities of specific endpoints, and (3) the 'definitions' section outlines the common data models reused throughout the API. Our focus is on computing the similarity of specifications at the endpoint level, so we primarily consider the 'paths' and 'definitions' sections. Figure 2 illustrates how the 'paths' section is converted into endpoint specifications, while the 'definitions' section is parsed to create a dictionary where keys represent data model names and values represent their properties (i.e. input and output data for the model).

We propose to integrate information from the 'paths' and 'definitions' sections to represent each endpoint in a tree structure, as depicted in Fig. 2. This approach captures the detailed hierarchical information within a specification and supports context-aware recommendations. For example, a term used in a parameter may have a different significance compared to its usage in a data model, enabling more precise recommendations based on a user's query. The top-level node of the tree represents the endpoint name, while its child nodes represent the operations defined for that endpoint. The leaf nodes contain essential content, such as description text, variable names, and the properties of data models referenced using the *$ref* token. During preprocessing, two types of key components are extracted from the leaf nodes of an endpoint tree:

1. Variable names (e.g. parameters, response names, properties) that are used to generate tree path tokens. These variable names are prefixed with the relevant parent tokens to create tree path tokens (e.g. 'songID' becomes 'parameters_songID').

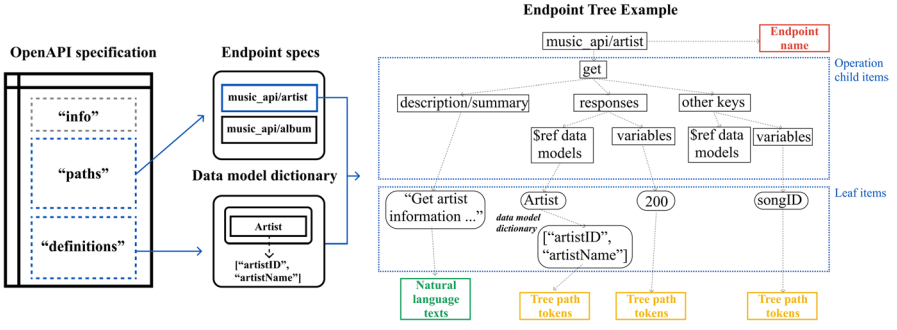


Fig. 2 Tree structure visualization of an endpoint specification

2. Natural language text (i.e. summaries and descriptions of operations). Note that descriptions of responses or parameters were not included, as they are often standardized (e.g. 'OK' for response 200) and thus not particularly useful for characterizing the endpoint.

To prepare the database for API-Miner, a collection of endpoint trees is created from the available OpenAPI specifications. Endpoint names, tree path tokens, and the natural language text from each endpoint tree are processed to generate features that characterize each unique endpoint. This process enables API-Miner to retrieve endpoint specifications that are relevant to a user's query (i.e. their current endpoint specification draft). The details of this process are explained in Sect. 4.2.

4.2 Probabilistic log-linear model for API endpoint ranking

Despite the existence of best practice guidelines, there is often significant variability in how OpenAPI specifications are authored, largely due to differences in individual writing styles. For instance, some developers might emphasize descriptions, while others may focus primarily on specifying inputs and outputs. To address this variability, we designed a probabilistic log-linear model that integrates multiple similarity signals. Probabilistic log-linear models have been successful in various fields, including Natural Language Processing (NLP) (Yang and Eisenstein 2013; Och and Ney 2002; Papineni et al. 1997) and Information Retrieval (IR) (Van Gysel et al. 2016), particularly in point-wise Learning To Rank (LTR) (Liu 2009). These models are well-suited for our task of endpoint ranking because they can accommodate a diverse range of feature representations to estimate the probability of relevance. The probability $P(e_i|e_j)$ of a match between two endpoint specifications is defined in Eq. 1 as a linear combination of features and weights derived from *tree path*, *fuzzy string*, description *text*, and endpoint specification *quality*.

$$P(e_i|e_j;w) = \frac{\exp(\sum_{k=1}^{F-1} w_k f_k(e_i, e_j) + w_F f_q(e_j))}{\sum_{i=1}^{N_E} \exp(\sum_{k=1}^{F-1} w_k f_k(e_i, e_j) + w_F f_q(e_i))} \tag{1}$$

Here, e_i and e_j represent the sets of operations (illustrated in Fig. 2) associated with endpoints i and j. The $f_k(.)$ are F *feature functions* that assess different aspects of the endpoints based on their operations' properties (such as text, tree path tokens, and endpoint names), while $f_q(.)$ accounts for the quality of the specifications. The fusion weights are denoted by $\{w_1, w_2, w_3, w_4\}$.

$$\hat{P}(e_i|e_j) = \frac{P(e_i|e_j)}{\max_k P(e_k|e_j)} \tag{2}$$

During inference, we apply Eq. 2 to rank candidate endpoints e_i given a query endpoint e_j. This ranking is determined by computing $\hat{P}(e_i|e_j)$ for all endpoints e_i in the database and ordering the resulting likelihoods in descending order. To enhance

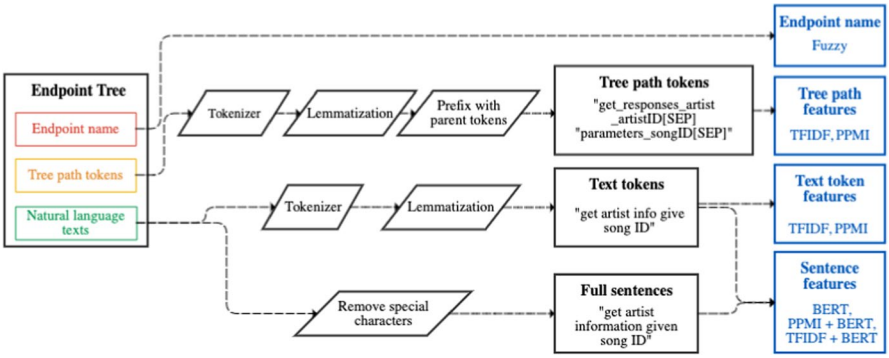


Fig. 3 Featurization of endpoint specifications

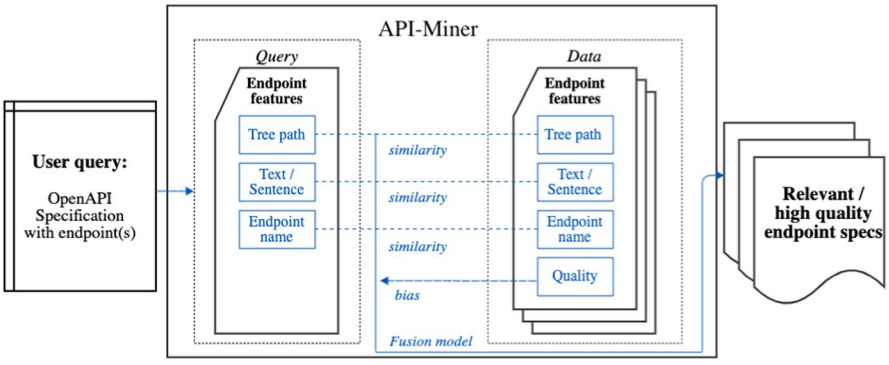


Fig. 4 Overall diagram of API-Miner

readability of small likelihoods without altering the ranked order, we normalize these likelihoods by the maximum observed matching likelihood.

For our API-Miner log-linear probabilistic model, we define four (F=4) feature functions. Three of these functions assess the similarity between query and data endpoint specifications, forming our *query-dependent* features. These similarities are measured across: (1) tree path tokens (Sect. 4.2.1), (2) endpoint names (Sect. 4.2.2), and (3) operational descriptions (Sect. 4.2.3). We also incorporate a fourth feature function, which is *query-independent* and evaluates the quality of the specifications within the database (Sect. 4.2.4). Ideally, the fusion weights $\{w_1, w_2, w_3, w_4\}$ could be optimized by minimizing the negative log-likelihood via gradient descent, $\hat{w} = argmin_w - \sum_{i=1}^{N} \log P(e_i|e_i^{'};w)$, based on a supervised signal of N relevant endpoint pairs $(e_i, e_i^{'})$. However, due to the scarcity of large-scale annotated data, we empirically set the weight of the

quality features (f_q) to 0.1, distributing the remaining 0.9 of the weights equally among the other features to prioritize the relevance of the retrieved endpoint. Sections 4.2.1 to 4.2.4 summarize the various methods we explored for evaluating the similarity and quality of endpoint specifications. The feature extraction methods, as well as the overall API-Miner workflow, are illustrated in Figs. 3 and 4.

4.2.1 Tree path token similarity

We consider the similarity of tree path tokens in our probabilistic model. Tree path tokens, which are generated from variable tokens, are extracted from endpoint trees by preprocessing (i.e. tokenizing, removing special characters, and lemmatizing) parameters and properties of referenced data models. A variable token is prefixed with its parent node name or a combination of the operation and response names to create a tree path token, such as *'parameters_songID'* or *'get_responses_200_Artist_artistName'*. We denote e^{tree} as the collection of tree path tokens t in endpoint e. We experiment with two techniques: first, Term Frequency - Inverse Document Frequency (TF-IDF) (Sparck Jones 1988), which counts word frequency to assess their relevance to a given document, and second, PMI (Church and Hanks 1990), where we convert the collection of tree path tokens into a numeric feature vector for endpoint similarity comparison. The TF-IDF weight for token t_i in endpoint e_j is defined in Eq. 3.

$$\begin{aligned}\text{TF-IDF}(t_i, e_j^{tree}) = &\frac{\#(t_i, e_j^{tree})}{\sum_{t_k \in V_T} \#(t_k, e_j^{tree})} \\ &\times \log(1 + \frac{N_E}{\sum_{j \in V_E} \mathbb{1}_{(\#(t_i, e_j^{tree}) \geq 1)}})\end{aligned} \tag{3}$$

where $\#(t_i, e_j^{tree})$ is the count of tree path token t_i in endpoint e_j, V_E is the endpoint and tree token vocabulary, respectively, and N_E is the number of endpoints. We represent an endpoint's tree path tokens e_j^{tree} as a vector $x_j \in \Re^{N_T}$ weighted by TF-IDF, where $x_j[i] = TF{-}IDF(t_i, e_j^{tree})$. To determine the similarity between endpoints' tree path tokens, we compute the cosine similarity between their TF-IDF vectors (Eq. 4).

$$f_1(e_i, e_j) = \cos(e_i^{tree}, e_j^{tree}) = \frac{x_i . x_j}{\sqrt{x_i^T x_i}\sqrt{x_j^T x_j}} \tag{4}$$

TF-IDF scoring is designed to highlight tokens that are frequently used within an endpoint specification but less common across the entire collection of N_E endpoint specifications. A limitation of TF-IDF is its inability to account for token correlations. For instance, the token *'parameters_firstName'* should have a higher matching score with the token *'parameters_lastName'* since both typically appear in similar contexts, compared to tokens that do not co-occur frequently. To capture token correlations, we compute the PPMI score between the N_T tree tokens in V_T. For endpoints $e \in V_E$ and their tree path tokens $t \in V_T$, we define the collection of all

possible tree token pairs by D. We denote $\#(t_i, t_j)$ as the count of the jth token co-occurring with the ith token in D. We further define $\#(t_i) = \sum_{t_j \in V_T} \#(t_i, t_j)$ and $\#(t_j) = \sum_{t_i \in V_T} \#(t_i, t_j)$. PPMI can then be expressed as in Eqs. 5, 6.

$$\text{PMI}(t_i, t_j) = \log \frac{P(t_i, t_j)}{P(t_i)P(t_j)} = \log \frac{\#(t_i, t_j).|D|}{\#(t_i)\#(t_j)} \tag{5}$$

$$\text{PPMI}(t_i, t_j) = \max(\text{PMI}(t_i, t_j), 0) \tag{6}$$

We construct the PPMI matrix $Q \in \mathfrak{R}^{N_T \times N_T}$, where $Q_{ij} = \text{PPMI}(t_i, t_j)$. We utilize matrix Q in a modified version of cosine similarity (Eq. 7) that assigns higher similarity to endpoints sharing semantically related tokens as defined by PPMI.

$$f_1(e_i, e_j) = cos(e_i^{tree}, e_j^{tree})_Q = \frac{x_i Q x_j}{\sqrt{x_i Q x_i}\sqrt{x_j Q x_j}} \tag{7}$$

where x_i, x_j are vectors of integer tree token frequencies.

Before generating TF-IDF and PPMI features, we filter out tree path tokens that appear in fewer than 10 endpoint specifications in the database. This step is taken to eliminate very unique tokens and prevent overfitting, which is particularly important for PPMI features known to be biased towards rare terms (Turney and Pantel 2010). In this paper, models that utilize features from tree path tokens are referred to as *'Models using tree path features'*.

4.2.2 Natural language text similarity

Endpoints can also be characterized by the natural language text found in the *'description'* and *'summary'* sections of their operations. These texts can be represented in three ways: (1) by the entire text, (2) by focusing solely on the keyword tokens within the text, or (3) by combining both methods.

Firstly, we can use pre-trained language models such as BERT (Devlin et al. 2018) and SENT-BERT (Reimers and Gurevych 2019) to generate representations of the entire text. This method allows us to obtain sentence-level embeddings based on the first 512 tokens (due to the context length limitation of BERT and SENT-BERT), thereby capturing the contextual representations of the text. SENT-BERT employs a siamese architecture, consisting of two BERT models that share the same weights, with the key difference being that SBERT processes sentence pairs during training. However, as discussed in Sect. 3.2, pre-trained language models face challenges in representing domain-specific low-resource terms.

Secondly, we can extract keyword tokens by preprocessing the entire text (i.e. tokenizing, removing stop words and special characters, and lemmatizing). We can then generate TF-IDF and PPMI features for these keyword tokens, as detailed in Sect. 4.2.1, allowing us to represent domain-specific terms that may not be well-represented in pre-trained language models. To avoid the bias toward rare tokens

when constructing the PPMI matrix, we filter out tokens that appear in fewer than 15 endpoints in the database.

Thirdly, we explore combining the above approaches to create 'enriched text features'. In essence, this involves concatenating the contextual text representations from pre-trained language models with keyword token representations obtained through TF-IDF or PPMI. Several approaches were investigated to linearly project the keyword and entire text features into a shared space before concatenation. We experimented with applying truncated SVD to the sparse keyword embeddings, reducing their dimensionality to a level where the total explained variance is 0.95. Following this, we applied canonical correlation analysis (CCA) to linearly project the keyword and entire text features into the same space before concatenation. CCA has been successfully applied in fusing multi-modal data (Sargin et al. 2007) and cross-language tasks in natural language processing (Haghighi et al. 2008), (Vinokourov et al. 2002). The output dimension for the CCA transformation is set to match the original dimension of BERT_SENT (384) or BERT (768). The projection is performed by maximizing the correlation (Eq. 8) between pre-trained embeddings X and keyword embeddings Y. The final features are defined in Eq. 9.

$$(u', v') = arg \max_{u,v} \frac{u^T X^T Y v}{\sqrt{(u^T X^T X u)(v^T Y^T Y v)}} \tag{8}$$

$$e^{text} = concatenate(u' X, v' Y) \tag{9}$$

where u' is the projection matrix for pre-trained embeddings X, and v' is the projection matrix for keyword embeddings Y.

We hypothesized that concatenating these features would allow us to retain the contextual understanding provided by pre-trained language models (through BERT / SENT-BERT features of entire texts) while also adequately representing domain-specific terms (through PPMI / TFIDF features of keyword tokens). These features are compared using cosine similarity (Eq. 4), and we represent this similarity as $f_2(e_i, e_j) = cos(e_i^{text}, e_j^{text})$. In this paper, models that utilize text features are referred to as *'Models using text features'*.

4.2.3 Fuzzy matching of endpoint names

The Levenshtein or string edit distance (Levenshtein 1966) measures the number of edits (insertions, deletions, or substitutions) required to transform one string into another. The more operations needed, the greater the distance between the strings. We apply the Levenshtein distance to compute the fuzzy matching between API endpoint names,[2] where $f_3(e_i, e_j) = lev(e_i^{name}, e_j^{name})$, and $lev(.)$ is a function that computes the Levenshtein distance. In this paper, models that employ fuzzy matching of endpoint names are referred to as *'Models using fuzzy matching'*.

[2] The fuzzywuzzy library [43] was used to compute the fuzzy match scores.

4.2.4 Quality score biasing

As a final signal, we bias the recommendation system to prioritize retrieving higher-quality specification components over lower-quality ones. Each OpenAPI specification in the database was assigned a numeric quality score based on OpenAPI 2.0 best practices [3]. Specifically, both the *'info'* and *'paths'* sections of the specification were evaluated. For both sections, the score is calculated based on whether all required keys are present and whether the data types of the values associated with the expected keys match those outlined in Table 1. The scores are computed using Eqs. 10–12.

$$f_q(e_i) = \lambda_1 q(e_i^{paths}) + \lambda_2 q(e_i^{info}) \tag{10}$$

We empirically set $\lambda_1 = 0.7, \lambda_2 = 0.3$ in our experiments to give greater emphasis on the quality of the paths. $q(e_i^{paths})$ and $q(e_i^{info})$ are computed using Eqs. 11–12. Equation 12 calculates the quality of the *info* and *paths* fields at depth d. The quality at depth d is the mean of the quality at depth $d-1$. Equation 11 computes the quality of *info* and *paths* at the lowest depth of 1. Note that for $q(e_i^{info})$, quality was computed up to depth 1 (i.e. based on keys and values contained directly in the *info* field) and thus only Equation 11 was needed to compute the quality score. For $q(e_i^{paths})$, quality was computed up to depth 3. Specifically, Eq. 11 was used to compute the quality of each operation containing responses with descriptions (depth 1). Then, the qualities of the operations were averaged to compute the quality of an endpoint path (depth 2), and then the qualities of the endpoints were averaged to compute $q(e_i^{paths})$ (depth 3) using Eq. 12.

$$q(e_i^1) = \begin{cases} 0 & \text{if } \#(e^1, K_{required}) < N_{K_{required}} \\ \frac{\#(e^1, K_{expected}, T)}{\#(e^1, K_{expected})} & \text{if } \#(e^1, K_{required}) = N_{K_{required}} \end{cases} \tag{11}$$

where $\#(e^1, K_{expected}, T)$ counts the number of times the data type of value corresponding to expected keys (i.e. keys that are in $K_{expected}$) present in e^1 match the expected data types in T, $\#(e^1, K_{required})$ counts the number of required keys in set

Table 1 Table to derive quality of OpenAPI specification

Parent key	Required child keys	Expected child key - value data type pairs
"info"	["title", "version"]	[("title": str), ("description": str), ("termsOfService": str), ("contact": dict), ("license": dict), ("version": str)]
"get", "put", "post", "delete", "options", "head", "patch"	["responses"]	[("tags": list), ("summary": str), ("description": str), ("externalDocs": dict), ("operationId": str), ("consumes": list), ("produces": list), ("parameters": dict), ("responses": dict), ("schemes": list), ("deprecated": bool), ("security": dict)]

$K_{required}$, $\#(e^1, K_{expected})$ is the total number of expected keys that are present in e^1, and $N_{K_{required}}$ counts the total number of required keys.

$$q(e_i^d) = \overline{q(e_i^{d-1})}, \qquad \text{if d} > 1 \tag{12}$$

where d is the depth of the current component relative to the original field (i.e. *info* and *paths*), and $\overline{q(e_i^{d-1})}$ is the mean of all $q(e_i)$ at depth $d-1$.

5 Evaluation

5.1 Dataset availability

The dataset was sourced from the APIS.guru GitHub repository [44], which houses publicly accessible OpenAPI specifications. At the time of our experiments, this repository included 3,699 OpenAPI specifications. For the scope of this paper, we incorporated 3,255 specifications written in OpenAPI version 2.0 into our dataset and assigned each a quality score ranging from 0 to 1. The quality scores varied from 0.3 to 1, with an average score of 0.85. From the dataset, a total of 11,778 unique endpoints were extracted to generate distinct endpoint specifications. Each endpoint specification, along with the essential components derived from its tree representation (such as tree path tokens and text), was stored in our database. In cases where multiple OpenAPI specifications shared the same endpoint name (for example, due to different versions of the same API), the key components were concatenated and updated within the database. This database was subsequently used to generate the feature vectors needed to characterize each endpoint specification in the database and to train the fusion models that were evaluated. The dataset used in this study can be accessed at the following public repository: https://github.com/APIs-guru/openapi-directory. This public repository is an open-sourced, community-driven project aimed at gathering a comprehensive, standards-compliant, and up-to-date directory of machine-readable OpenAPI specifications.

5.2 Models evaluated

The featurization methods explored for each similarity signal are outlined in Table 2. Every possible combination of these featurization methods was evaluated to

Table 2 Featurization methods explored

Similarity approach	Featurization method
Tree path	[TFIDF, PPMI]
Text	[TFIDF, PPMI, BERT, SENT_BERT, TFIDF + BERT, TFIDF + SENT_BERT, PPMI + BERT, PPMI + SENT_BERT]
Fuzzy	N/A

determine the optimal configuration for the fusion model. Among the various fusion models explored, several models that resembled existing works were chosen as baselines. To replicate approaches that utilized fuzzy matching, the model that solely computes the fuzzy matching of endpoint names to assess similarity was selected as a baseline. To emulate approaches that leveraged the semantic similarity of natural language texts within the specification using pre-trained language models, models that only use BERT or SENT_BERT to represent entire natural language texts were chosen as baselines.

5.3 Retrieval tasks

5.3.1 Experiment setup

For the evaluation process, 1000 randomly chosen endpoint specifications were altered and used as queries. The recall values for each fusion model were then calculated using Eq. 13:

$$Recall@i = \frac{TP_i}{N} \tag{13}$$

Where TP_i represents the number of instances where the original endpoint appears within the top $i = 1, 5, 10$ recommendations, and N is the total number of queries evaluated.

Assessing the recommendation system's capability to retrieve the original sample using a modified version as the query is a standard evaluation approach commonly employed in related studies (Luan et al. 2018). Consequently, we tested API-Miner's ability to retrieve the original endpoint specification when provided with its masked or mangled version. The procedure to generate queries for masked and mangled retrieval is illustrated in Fig. 5, where only the 'paths' section is depicted for clarity. Both masking and mangling modifications begin by eliminating 50% of the definitions, operations, and responses in the specification. Following this, the remaining content and the endpoint name are altered either through masking or mangling. Masking involves omitting certain parts of the specification. Specifically, 50% of the properties under each definition were removed, and 50% of the tokens in each operation's description or summary sections were deleted. Additionally, endpoint names were masked by randomly removing 30% of their characters. As shown in Fig. 5, the original endpoint name is masked, and randomly chosen properties and tokens are

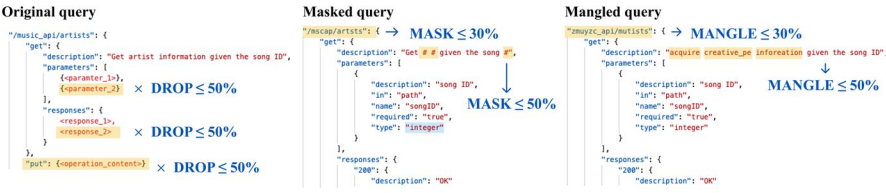


Fig. 5 Modification of an endpoint specification for retrieval tasks

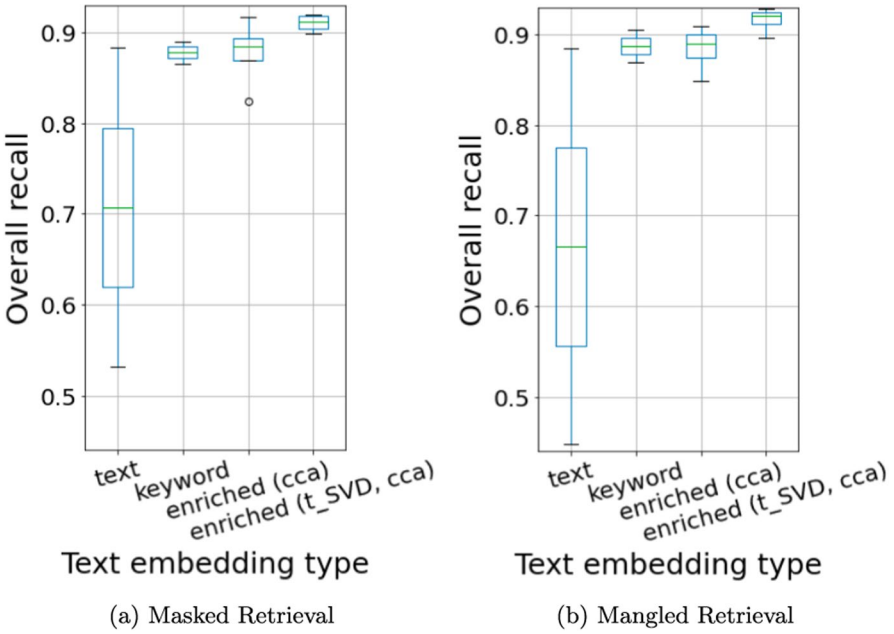


Fig. 6 Overall recall obtained from models using only text features, categorized by type of text feature used. Note that there were 2 combinations of text (i.e. BERT, BERT_SENT) and keyword embeddings (i.e. TF-IDF, PPMI) and 4 combinations of enriched embeddings evaluated

omitted. For clarity, the removed tokens were replaced with a special character ('#'), which is eliminated during preprocessing.

The mangling process involves replacing specific tokens with their misspelled versions or synonyms (identified using NLTK's WordNet library (Bird et al. 2009) (Fellbaum 1998)). The misspellings were simulated by substituting a random character in the word with another random letter. Specifically, 50% of the property names under each definition were mangled, and 50% of the tokens in each operation's description or summary were altered. Additionally, endpoint names were mangled by randomly replacing 30% of their characters with other random characters. As shown in Fig. 5, the original endpoint name is mangled, and randomly selected tokens are replaced with misspelled versions (e.g., "information" to "inforeation") or synonyms (e.g., "artist" to "creative_person").

5.3.2 Results

Table 3 presents the results of the retrieval tasks, categorized by the top-performing models. By comparing the performance of different single fusion models, we can evaluate the effectiveness of various signals (i.e. tree features, text features, etc.) in retrieving relevant endpoint specifications. It is evident that models utilizing only text features significantly outperform those that rely solely on tree or fuzzy features. Further analysis of models based purely on text features, as depicted in Fig. 6,

Table 3 Retrieval task results

	Model		Masked Retrieval			Mangled Retrieval			Combined
	Approach	Featurization	R@1	R@5	R@10	R@1	R@5	R@10	Average recall
Baseline	Text	BERT	0.453	0.555	0.587	0.379	0.467	0.497	0.490
	Fuzzy	N/A	0.426	0.695	0.782	0.416	0.689	0.768	0.629
	Text	SENT_BERT	0.763	0.928	0.958	0.762	0.930	0.963	0.884
Single fusion	Tree	TFIDF	0.505	0.759	0.830	0.516	0.768	0.834	0.702
	Text	PPMI + SENT_BERT (t_SVD + CCA)	0.828	0.958	0.969	0.842	0.963	0.979	0.923
Double fusion	Tree, Fuzzy	TFIDF N/A	0.692	0.888	0.932	0.682	0.877	0.926	0.833
	Tree, Text	PPMI TFIDF + BERT_SENT (t_SVD + CCA)	0.856	0.974	0.982	0.877	0.982	0.992	0.944
	Text, Fuzzy	TFIDF + BERT_SENT (t_SVD + CCA) N/A	0.9	0.988	0.994	0.904	0.988	0.996	0.962
Triple fusion	Tree, Text Fuzzy	PPMI TFIDF N/A	0.896	0.991	0.995	0.912	0.993	0.999	0.964
	Tree, Text Fuzzy	PPMI TFIDF + BERT_SENT (t_SVD + CCA) N/A	**0.910**	**0.989**	**0.993**	**0.924**	**0.996**	**0.999**	**0.969**

Bold indicates the highest performing variant of the model

indicates that enriched representations of natural language text lead to superior results. Specifically, the best performance in both retrieval tasks is achieved by first applying truncated SVD to the keyword embeddings, followed by CCA to project the keyword and entire text embeddings linearly before concatenation. Overall, there is a clear trend of improved performance from single to triple fusion models, with the highest combined recall of 0.969 achieved by a triple fusion model. This finding supports our hypothesis that incorporating multiple sources of similarity enhances performance.

5.4 Pooling-based user study

5.4.1 Experiment setup

The pooling user study was designed to assess the subjective usefulness of API-Miner for users creating OpenAPI specifications. In this experiment, we aimed to simulate a typical scenario where a user begins developing an endpoint specification and uses this early draft as a query to API-Miner to retrieve relevant endpoints. To create realistic early draft queries, we enlisted 4 developers who each had at least a Master's degree in a quantitative discipline and an average of 7.5 years of industrial programming experience. By selecting annotators with similar technical expertise to potential users, we sought to evaluate the tool's relevance and utility. Prior to the study, the developers were given detailed examples to familiarize them with the specific version of the OpenAPI specification framework being used.

Each annotator was randomly assigned a task to design an endpoint by drafting an early version of an OpenAPI specification. These task prompts were automatically generated by randomly selecting endpoints from the database and extracting: (1) API summary (including "title," "summary," and "description" from the "info" section of the specification), and (2) the task description (including "description" and "summary" of the endpoint's operations). To ensure that annotators had sufficient information to craft their queries, only tasks containing at least 10 tokens in the API summary and task description were considered. Annotators were instructed to design at least one of the tasks (e.g., operation descriptions) included in the prompt.

The top 10 models with the best average recall from retrieval tasks, along with baseline models, were evaluated in this user study. Given an early draft query created by the annotator, each model generated its top 5 recommendations, resulting in a pool of recommended endpoints per query. The usefulness of the recommendation system was defined by its ability to retrieve endpoint specifications that were beneficial given the user query. Usefulness was categorized into 4 levels: (0) not useful, (1) minimally useful, (2) slightly useful, and (3) highly useful. An endpoint rated 0 had no relevance to the query's functionality or content. Endpoints rated 1 or 2 contained related functionality or topics. An endpoint rated 3 had useful information or closely matched the content of the query. By employing a four-point rating scale, we ensured that the annotator could not give a neutral response, thereby

clearly classifying whether an endpoint contained useful information or not. To mitigate annotator bias (e.g., some annotators consistently giving high or low scores), each annotator's ratings were normalized from 0 to 1 using linear min-max scaling. A threshold of 0.5 was then applied to convert the normalized scores into binary labels: 0 (not useful to the query) or 1 (useful to the query). Overall, 846 samples (i.e., query and retrieved endpoint pairs) were annotated using a pooling strategy across annotators, allowing us to calculate precision, recall, and F1 for each model using Eqs. 14–16.

$$Precision = \frac{TP}{TP + FP} \tag{14}$$

$$Recall = \frac{TP}{TP + FN} \tag{15}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{16}$$

Where TP represents the number of useful endpoints recommended by the model, TP+FP is the total number of endpoints suggested by the model, and TP+FN indicates the total number of useful endpoints available in the pool.

To ensure a sufficient level of agreement among annotators, we generated 5 early draft queries by extensively modifying random endpoint specifications from the database. Specifically, all sections except for the descriptions and summaries of operations and parameters were removed, and 50% of the operations and parameters were eliminated. For sections with more than 5 tokens, 40% of the tokens in both the description and summary sections of operations and parameters were masked. Finally, only the last 30% of the characters in the endpoint names were retained, with the remainder being discarded. This process produced a total of 150 samples (i.e. query-retrieved endpoint pairs), which were subsequently annotated by each participant. The annotators achieved a Fleiss kappa score of 0.485, indicating moderate agreement among them (Landis and Koch 1977). It is worth noting that this level of agreement may be due to the fact that each annotator has a unique perspective on what an API endpoint should look like, making moderate agreement a reasonable expectation in this context.

5.4.2 Results

Table 4 reports the top 3 API-Miner and baseline models from the user study. We observe that user study results are consistent with the retrieval task results, in which the top three models from both evaluations match. Overall, the best performing API-Miner model surpasses the best performing baseline model by 2.5 percentage points (pp) in recall, 3.5 pp precision and 3.2 pp F1.

Table 4 User study results on top 3 API-Miner models and baseline models

	Approach	Featurization	Recall	Precision	F1
Baseline	Fuzzy	N/A	0.240	0.154	0.188
	Text	BERT	0.335	0.220	0.266
	Text	BERT_SENT	**0.665**	**0.440**	**0.530**
API-Miner	Text Fuzzy	TFIDF + BERT_SENT (t_SVD + CCA) N/A	0.665	0.451	0.538
	Tree, Text Fuzzy	PPMI TFIDF + BERT_SENT (t_SVD + CCA) N/A	0.680	0.453	0.544
	Tree, Text Fuzzy	PPMI TFIDF N/A	**0.690**	**0.475**	**0.562**

5.5 Discussion

In the retrieval tasks, we assessed API-Miner's capability to retrieve the original endpoint from a synthetically modified version and uncovered several noteworthy insights. Our findings demonstrate that API-Miner is effective in handling syntactically invalid specifications, highlighting its practical value for developers making real-time queries. First, we observed that natural language text provides the most significant signal for determining endpoint relevance. Additionally, enriched text features consistently outperformed other text features across all evaluation tasks. The combination of keyword and full-text features offers an enriched representation of the natural language text in API specification documents, particularly addressing low-resource and domain-specific terms. Furthermore, applying truncated SVD to the sparse keyword features before projecting them using CCA proved to yield the most meaningful representations of natural language text. We also noted that triple fusion models leveraging TF-IDF text features and TF-IDF + BERT_SENT features performed similarly across all evaluation tasks. This suggests that, within the scope of OpenAPI specification matching and recommendation, the benefits of rich representations from large pre-trained language models are limited. Overall, we conclude that the fusion of tree, text, and fuzzy similarity approaches produces the most effective model for OpenAPI recommendations, supporting our hypothesis that a probabilistic log-linear model effectively optimizes recommendation performance.

During the user study, we evaluated API-Miner's usefulness in a realistic scenario. Developers were given a task prompt, asked to draft an early version of an endpoint specification, and then evaluate the usefulness of the endpoints retrieved by each model. The results from the user study aligned with those from the retrieval tasks, with the best performance yielding a recall of 0.690, precision of 0.475, and F1 score of 0.562.

6 Conclusion

6.1 Threats to validity and limitations

There are several threats to the validity of our findings, particularly related to the setup of the user study. To keep the annotation workload manageable, we used a pooling-based approach. While pooling is a widely accepted method for building test collections in Information Retrieval (Jones et al. 1975; Zobel 1998), it can introduce biases, especially with larger collection sizes (Buckley et al. 2006). Additionally, reaching a consensus in API annotation can be challenging due to varying developer preferences, which contributed to the relatively low/moderate agreement among annotators in our user study (i.e. kappa = 0.485). To mitigate these limitations, we also incorporated an automated evaluation method (i.e. retrieval tasks).

6.2 Summary

This paper outlines the key principles of API-Miner and demonstrates its ability to recommend relevant and high-quality OpenAPI specification components to users. We report strong performance in both retrieval tasks and user studies, showing that the optimal configuration of API-Miner can surpass the baseline by 15.4% in Recall@1 and 3.2% in F1 score. By introducing well-defined methods for parsing and extracting features from OpenAPI specifications and utilizing a novel linear-late fusion similarity algorithm, API-Miner represents pioneering research in the field of OpenAPI specification recommendation.

We are eager to build upon this research and explore several avenues for future work. Firstly, we aim to develop machine learning algorithms to learn the optimal weights for the fusion model. Instead of using grid search to set the weights for the similarity approaches in the model, we could use hill-climbing optimization techniques to further enhance recommendation performance. Additionally, one limitation of the current implementation of API-Miner is its exclusive support for OpenAPI version 2.0. We plan to develop a more version-agnostic preprocessing module capable of handling different versions of OpenAPI specifications, and potentially other types of web service documents. Finally, although we employed a small language model-BERT-to impart some semantic understanding to API-Miner, traditional techniques proved more effective due to the narrowness of the domain. Given API-Miner's design as a lightweight and cost-efficient system, exploring the potential of Large Language Models (LLMs) with a larger computational and budgetary allowance presents an intriguing direction for future research.

References

APIs-guru: Wikipedia for Web APIs. Directory of REST API definitions in OpenAPI 2.0/3.x format (2022). https://github.com/APIs-guru/openapi-directory

Aguilera U, Abaitua J, Diaz J, Bujan D, Ipina DLD (2007) A semantic matching algorithm for discovery in uddi. International Conference on Semantic Computing. https://doi.org/10.1109/icsc.2007.43

Bahdanau D, Bosc T, Jastrzębski S, Grefenstette E, Vincent P, Bengio Y (2018) Learning to Compute Word Embeddings On the Fly. https://openreview.net/forum?id=B1CNpYg0-

Bai L, Liu M (2011) Fuzzy sets and similarity relations for semantic web service matching. Computers & Mathematics with Applications 61(8):2281–2286

Bird S, Klein E, Loper E (2009) Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. "O'Reilly Media, Inc.", ???

Buckley C, Dimmick D, Soboroff I, Voorhees E (2006) Bias and the limits of pooling. In: Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval. SIGIR '06, pp. 619–620. Association for Computing Machinery, New York, NY, USA

Church KW, Hanks P (1990) Word association norms, mutual information, and lexicography. Comput Linguist 16(1):22–29

Devlin J, Chang M, Lee K, Toutanova K (2018) BERT: pre-training of deep bidirectional transformers for language understanding. CoRR **abs/1810.04805** https://arxiv.org/abs/1810.04805arXiv:1810.04805

Fellbaum C (1998) opeWordNet: An Electronic Lexical Database. Bradford Books, ???

Fielding RT, Taylor RN (2000) Architectural styles and the design of network-based software architectures. PhD thesis, University of California. AAI9980887

Fielding RT (2000) Architectural styles and the design of network-based software architectures. PhD thesis, University of California, Irvine. Doctoral dissertation

Fletcher K (2019) Regularizing matrix factorization with implicit user preference embeddings for web API recommendation. In: 2019 IEEE International Conference on Services Computing (SCC), pp. 1–8. https://doi.org/10.1109/SCC.2019.00014

Hadley MJ (2006) Web application description language (WADL). Inc, Sun Microsystems

Haghighi A, Liang P, Berg-Kirkpatrick T, Klein D (2008) Learning bilingual lexicons from monolingual corpora. In: Proceedings of ACL-08: Hlt, pp. 771–779

Initiative O (2021) OpenAPI: Open API Specification. https://github.com/OAI/OpenAPI-Specification

Initiative O (2021) OpenAPI: Open API Specification, Version 3.1.0.. https://github.com/OAI/OpenAPI-Specification. https://github.com/OAI/OpenAPI-Specification

Jones KS, Van Rijsbergen CJ, Research BL, Department D (1975) Report on the Need for and Provision of an 'ideal' Information Retrieval Test Collection. British Library Research and Development reports. University Computer Laboratory, ??? . https://books.google.co.uk/books?id=cuGnSgAACAAJ

Jungmaier J, Kassner N, Roth B (2020) Dirichlet-smoothed word embeddings for low-resource settings. CoRR **abs/2006.12414** https://arxiv.org/abs/2006.12414arXiv:2006.12414

Khorasgani RR, Stroulia E, Zaiane OR (2011) Web service matching for restful web services. In: 2011 13th IEEE International Symposium on Web Systems Evolution (WSE), pp. 115–124. IEEE

Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. Biometrics 33(1):159–174

Levenshtein VI (1966) Binary codes capable of correcting deletions, insertions and reversals. Soviet Physics Doklady 10(8):707–710 Doklady Akademii Nauk SSSR, V163 No4 845-848 1965

Liu T-Y (2009) Learning to rank for information retrieval. Found Trends Inf Retr 3(3):225–331. https://doi.org/10.1561/1500000016

Luan S, Yang D, Sen K, Chandra S (2018) Aroma: Code recommendation via structural code search. CoRR **abs/1812.01158** https://arxiv.org/abs/1812.011581812.01158

Moreo A, Esuli A, Sebastiani F (2021) Word-class embeddings for multiclass text classification. Data Min Knowl Disc 35(3):911–963. https://doi.org/10.1007/s10618-020-00735-3

Och FJ, Ney H (2002) Discriminative training and maximum entropy models for statistical machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 295–302. Association for Computational Linguistics, Philadelphia, Pennsylvania, USA. https://doi.org/10.3115/1073083.1073133 . https://aclanthology.org/P02-1038

Papineni K, Roukos S, Ward T (1997) Feature-based language understanding. In: EUROSPEECH

Peng C, Goswami P, Bai G (2018) Fuzzy matching of OpenAPI described REST services. Procedia Computer Science 126:1313–1322. https://doi.org/10.1016/j.procs.2018.08.081

Pinter Y, Guthrie R, Eisenstein J (2017) Mimicking word embeddings using subword RNNs. Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing https://doi.org/10.18653/v1/d17-1010

Qi L, He Q, Chen F, Dou W, Wan S, Zhang X, Xu X (2019) Finding all you need: Web APIs recommendation in web of things through keywords search. IEEE Transactions on Computational Social Systems 6(5):1063–1072. https://doi.org/10.1109/TCSS.2019.2906925

Recommendation W (2007) WSDL: Web Services Description Language. https://www.w3.org/TR/wsdl/

Reimers N, Gurevych I (2019) Sentence-bert: Sentence embeddings using siamese bert-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, ???. https://arxiv.org/abs/1908.10084

Richardson L, Ruby S (2007) RESTful Web Services. O'Reilly Media Inc, Sebastopol, CA, USA

Sargin ME, Yemez Y, Erzin E, Tekalp AM (2007) Audiovisual synchronization and fusion using canonical correlation analysis. IEEE Trans Multimedia 9(7):1396–1403

Schick T, Schütze H (2019) Attentive mimicking: Better word embeddings by attending to informative contexts. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 489–494. Association for Computational Linguistics, Minneapolis, Minnesota. https://doi.org/10.18653/v1/N19-1048 . https://aclanthology.org/N19-1048

Seatgeek: Fuzzy String Matching in Python (2018). https://github.com/seatgeek/fuzzywuzzy

Sohan SM, Anslow C, Maurer F (2015) A case study of web API evolution. In: 2015 IEEE World Congress on Services, pp. 245–252. https://doi.org/10.1109/SERVICES.2015.43

Sparck Jones K (1988) A Statistical Interpretation of Term Specificity and Its Application in Retrieval, pp. 132–142. Taylor Graham Publishing, GBR

Stroulia E, Wang Y (2005) Structural and semantic matching for assessing web-service similarity. International Journal of Cooperative Information Systems 14(04):407–437. https://doi.org/10.1142/s0218843005001213

Swagger: OpenAPI Specification version 2.0 (2014). https://swagger.io/specification/v2/

Swagger: OpenAPI Specification Version 2.0. (2014). https://swagger.io/specification/v2/. https://swagger.io/specification/v2/

Thung F, Oentaryo RJ, Lo D, Tian Y (2017) Webapirec: Recommending web APIs to software projects via personalized ranking. IEEE Transactions on Emerging Topics in Computational Intelligence 1(3):145–156. https://doi.org/10.1109/TETCI.2017.2699222

Turney PD, Pantel P (2010) From frequency to meaning: Vector space models of semantics. J. Artif. Int. Res. 37(1):141–188

UDDI.org: UDDI: Universal Description, Discovery, and Integration (2000)

Van Gysel C, Rijke M, Worring M (2016) Unsupervised, efficient and semantic expertise retrieval. In: Proceedings of the 25th International Conference on World Wide Web. WWW '16, pp. 1069–1079. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE. https://doi.org/10.1145/2872427.2882974

Vinokourov A, Cristianini N, Shawe-Taylor J (2002) Inferring a semantic representation of text via cross-language correlation analysis. Adv Neural Inf Process Syst 15:1497–1504

Wang X, Wu H, Hsu C-H (2019) Mashup-oriented API recommendation via random walk on knowledge graph. IEEE Access 7:7651–7662. https://doi.org/10.1109/ACCESS.2018.2890156

Wang Y, Stroulia E (2003) Flexible interface matching for web-service discovery. In: Proceedings of the Fourth International Conference on Web Information Systems Engineering, 2003. WISE 2003., pp. 147–156. https://doi.org/10.1109/WISE.2003.1254478

Wsantos: APIs show Faster Growth Rate in 2019 than Previous Years (2019). https://www.programmableweb.com/news/apis-show-faster-growth-rate-2019-previous-years/research/2019/07/17

Yang Y, Eisenstein J (2013) A log-linear model for unsupervised text normalization. In: Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, pp. 61–72. Association for Computational Linguistics, Seattle, Washington, USA. https://aclanthology.org/D13-1007

Young Moon S, Kerr G, Silavong F, Moran SS (2024) Api-miner: an api-to-api specification recommendation engine. In: The 1st Workshop on Software Engineering Challenges in Financial Firms (FinanSE)

Zobel J (1998) How reliable are the results of large-scale information retrieval experiments? In: Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval. SIGIR '98, pp. 307–314. Association for Computing Machinery, New York, NY, USA

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Saeyoung Moon[1] · Gregor Kerr[1] · Fran Silavong[1] · Sean Moran[1]

✉ Sean Moran
sean.j.moran@jpmchase.com

Saeyoung Moon
saeyoung.moon@jpmchase.com

Gregor Kerr
gregor.kerr@jpmchase.com

Fran Silavong
fran.silavong@jpmchase.com

[1] JPMorgan Chase, 25 Bank Street, London E145JP, UK

Data Mining & Knowledge Discovery is a copyright of Springer, 2025. All Rights Reserved.